

The four building blocks every program is made from

A computer can only ever do a handful of truly different things: store a value, decide between paths, repeat work, calculate with operators, and reuse a named block of code. Master these and you can read or write any object-oriented program ever written.

What this key knowledge covers

This summary distils everything you need for **KK08 — The four building blocks every program is made from** in Programming. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

□ What the study design actually asks

The VCAA wording for this dot point lists the *features of an object-oriented programming language* you must be able to use and describe:

□ Work top-to-bottom for a full lesson, or jump via the rail. Interactive checks feed the score ring (bottom-right). Everything is saved in your browser — close the tab and come back later.

01 · The Control-Flow Engine

A running program is a single point of attention — the *program counter* — moving through your instructions. The three **control structures** are the only three shapes that path can take. Press play and watch the glowing token travel.

The fork is a **selection** (member → discount branch, guest → straight on). The circular return path is an **iteration** that runs once per topping layer. Change the toggles and replay to see the path physically re-route.

Steps run one after another, top to bottom. The default — no keyword needed.

A condition picks *one* branch. if / elif / else.

A block repeats. for (counted) or while (condition).

02 · Variables — labelled boxes in memory

A *variable* is a **named box** that holds one value at a time. The name is the label on the box; the value inside can be swapped out as the program runs. The kind of value it holds (its *data type* — text, number, Boolean) was KK02; here we care about *using* it.

Think of the kiosk's order screen. Each thing it remembers is a variable:

An **assignment** uses = meaning "put the value on the right into the box on the left". It is *not* the maths "equals".

□ **Step the value through**

Watch one box, total, change as three lines run. Tap **Next line**.

03 · Control structures up close

The engine showed the shapes; here is the code and the analogy for each. Examiners love asking you to **name** the structure and **justify** why it suits the task.

① **Sequence — "follow the recipe in order"**

Instructions execute one after another, exactly as written. Order matters: pour tea *before* adding the lid.

② **Selection — "the fork in the road"**

A *condition* decides which branch runs. Use if for one test, elif for more, else for the catch-all.

③ **Iteration — "keep going until done"**

A block repeats. A **counted** loop (for) runs a known number of times; a **conditional** loop (while) runs until something becomes false.

04 · Operators — the verbs of code

Operators *combine or compare* values. The study design names three families. Mixing them up is one of the most common dropped marks, so nail the categories.

Do maths, give a **number**. + − * / // % **

Compare two values, give **True/False**. == != < > <= >=

Join True/False values. and · or · not

Priya's discount rule needs all three at once — arithmetic to build the total, relational to test it, logical to combine conditions:

□ **Operator playground**

Pick two values and an operator. Arithmetic returns a number; relational and logical return True / False. Try 9.10 > 8.00, then True and False.

This is a **relational** operator → returns True/False.

05 · The Function Factory

A **procedure**, **function** and **method** are all named, reusable blocks — but the exam wants you to tell them apart. Watch the machine and the difference clicks: it's about *what comes back* and *who it belongs to*.

06 · Trace the loop

“Desk-checking” — running code by hand and recording each variable — is a guaranteed exam skill. Fill the table for this counted loop, then check it. (3 marks to the ring.)

07 • Name that feature

Drag each snippet into the family it belongs to. On a phone, tap a snippet then tap a bin. (6 marks to the ring.)

08 • Decode the command word

VCAA marks the *verb*. “Identify” is worth less effort than “explain”; “justify” needs a reason. Match your answer length to the word.

Name it. One word or short phrase. “*Selection (an if-statement).*”

Say what it is **and** a feature of it. Two sentences.

Say what **and why/how** it works in this context. Cause → effect.

Give a **similarity and a difference** — both, explicitly, side by side.

Make a choice and defend it with a reason tied to the scenario.

Produce actual code or pseudocode . Syntax must be plausible and correct.

10 • Write it the P·E·L way

For “explain / justify” questions, structure beats waffle. **P**oint (name it) → **E**xample (from the scenario) → **L**ink (back to the question). Toggle a worked answer, then write your own and watch the meter.

“BobaBot should use a **selection** control structure (an if/elif/else) to choose the base price.”

Your turn. Q: *Explain why BobaBot needs a selection control structure to set the cup price.* Aim to hit all three P·E·L pips.

Type at least 12 words to score it.

11 • Quick six

Instant feedback. Each correct answer adds to the ring.

12 • VCE-style exam practice

Five questions, **20 marks**. Try each *before* revealing. Read the weak answer to see how marks slip, the strong answer for full marks, then self-mark — it feeds the ring.

13 • One-screen cheat sheet

- **Named box** holding a value that can change
- = is **assignment**, not maths equals
- Type comes from the value: text, number, Boolean
- **Sequence** — top-to-bottom, order matters
- **Selection** — if/elif/else, picks one branch
- **Iteration** — for (counted) / while (condition)
- **Arithmetic** → number: + - * / % **
- **Relational** → T/F: == != < > <= >=

- **Logical** join T/F: and · or · not
- **Function** — return s a value
- **Procedure** — does an action, no return
- **Method** — written object.name()
- = assign vs == compare
- **if** = selection (structure), > = operator
- logical (and) vs relational (>)
- Match **points to marks**
- Use **P•E•L** for explain/justify
- “Function” → always say “**returns**”

Key terms

Term	Meaning
Variables	named storage for a value that can change while the program runs
Control structures	sequence · selection · iteration / repetition
Operators	arithmetic · logical (AND/OR/NOT) · conditional / relational
Reusable blocks	procedures · functions · methods

Common traps & misconceptions

Exam trap

① = vs == One equals **assigns** (put a value in a box); two equals **compares** (relational). Writing if x = 5 is a bug and loses code marks.

Exam trap

② Function ≠ procedure If you say “function” you must mention it **returns** a value. Calling makeDrink() a function (it returns nothing) is wrong — it’s a procedure.

Exam trap

③ “Conditional operator” ≠ “if” Conditional/relational **operators** are the symbols > == !=. The if itself is a **control structure** (selection). Don’t swap the labels.

Exam trap

④ Logical vs relational > is relational (compares values). and / or / not are logical (join True/False). Examiners test the boundary deliberately.

Exam trap

⑤ for vs while Counted repetition → **for**. Repeat-until-condition → **while**. Justify your choice with “the count is known / not known”.

Exam trap

⑥ Sequence is still a structure Students forget plain top-to-bottom code *is* the sequence control structure. If asked to name all three, don't omit it.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. State

The three control structures available in an object-oriented language.

✗ A weak answer

— loses marks “Loops and if statements.” Only two ideas, both vaguely worded. “Loops” is informal for iteration; sequence is missing entirely. Caps at 1 mark.

✓ A strong answer

— full marks The three control structures are **sequence** (instructions run in order), **selection** (a condition chooses a branch, e.g. if/else) and **iteration** (a block repeats, e.g. for/while).

How the marks are awarded

- 1 mark per correctly named structure, to a max of 2 here (full list = 3 names).
- Accept ‘repetition’ for iteration.
- ‘Sequence’ must appear — most-missed item.

Q2. Identify

Priya writes: if isMember and total > 8.00:. Identify the two operator families used and explain what the whole condition evaluates to. Context: isMember is True and total is 9.10.

✗ A weak answer

— loses marks “It uses an if statement to check the member. It is true.” Names the control structure, not the *operators* asked for. Doesn’t identify ‘and’ (logical) or ‘>’ (relational). No working shown 1 mark of 3.

✓ A strong answer

— full marks > is a **relational (conditional)** operator and and is a **logical** operator. total > 8.00 9.10 > 8.00 True; True and True **True**, so the discount branch runs.

How the marks are awarded

- 1 mark: relational/conditional identified for >.
- 1 mark: logical identified for and.
- 1 mark: correct overall result True with brief working.

Q3. Explain

The difference between a **function** and a **procedure**, using a BobaBot example of each.

✗ A weak answer

— loses marks “A function does something and a procedure also does something. Like makeDrink and calculateTotal.” States they’re similar but never gives the distinguishing feature (return value). Examples aren’t tied to which is which. Around 1–2 marks.

✓ A strong answer

— full marks A **function returns a value** to wherever it was called: calculateTotal(7.50, 2) returns 9.10, which can be stored in total. A **procedure performs an action and returns nothing**: makeDrink() pours the tea and adds ice but hands no value back. The defining difference is the return.

How the marks are awarded

- 1 mark: function returns a value.
- 1 mark: procedure performs an action / no return.
- 1 mark each for a correct, matched BobaBot example (2).

Q4. Justify

BobaBot must keep pouring tea until the cup reaches 100 mL, adding 10 mL each pour. Justify which iteration structure suits this, then trace how many pours occur from an empty cup. Context: Each pour adds exactly 10 mL; start level is 0.

✗ A weak answer

— loses marks “Use a for loop 10 times because 100/10 is 10.” Reaches the right count but doesn’t justify *why* while suits an ‘until full’ goal. No trace of the changing level. Misses the justification marks ~2–3.

✓ A strong answer

— full marks A **while loop** suits it because we repeat *until a condition* (level < 100) is false rather than a fixed count — if the target changed, the same loop still works. Trace: level 0 10 20 ... 100, condition checked each pass; it pours on 0, 10, 20, 30, 40, 50, 60, 70, 80, 90 then stops at 100 **10 pours**. (A for loop also works here only because the count happens to be known.)

How the marks are awarded

- 1: identifies while (or for) as iteration.
- 2: justification references repeat-until-condition vs counted.
- 2: correct trace of level changing by 10.
- 1: correct final count of 10 pours.